AV2 · PACOTE 3 · PROBLEMA FORMAL

Janela de contexto como mochila 0/1

Entrada e custo

M = 200 candidatos · limite W

- Cada notícia inteira é um item (sem chunking)
- Valor **v**: score BM25 da consulta (≥ 0)
- Custo **c**: total de tokens do documento (> 0)

W = 500, 1.000, 2.000

orçamentos de tokens avaliados na grade

Restrição e objetivo

decisão 0/1: entra inteiro ou fica fora

- Restrição: soma dos tokens escolhidos ≤ W
- Objetivo: maximizar a soma dos scores BM25
- Corretude: maior valor total viável para (C, W)

max Σ v_i com Σ c_i ≤ W

ótimo entre os M = 200 candidatos da consulta

Contrato e fronteiras

validate(candidates, budget)

- Rejeita doc_id repetido, c ≤ 0, v < 0 ou W < 0
- Lista vazia, W = 0 ou nada cabe → seleção vazia
- Empates de valor: saída sempre determinística

Candidate → Selection

doc_ids em ordem crescente, valor, custo e ops

Corpus Folha/UOL (Kaggle, CC0) · 167.053 notícias (2015–2017) · amostras com **seed 42** (N = 10 mil e 50 mil)
Ficha técnica em **docs/FICHA_CORPUS.md** · Mediana de 238 tokens por artigo dispensa chunking

AV2 · PACOTE 3 · CONTINUIDADE E LINHAS DE BASE

O que veio da AV1 e as referências de comparação

Ponte com a AV1 (av2_candidates.py)

indexed_search + merge_sort → top M = 200

- **Mantido da AV1:** limpeza, tokenização, BM25 (k1 = 1,2 e b = 0,75), índice invertido hash e merge_sort
- **Acrescentado:** build_candidates corta nos 200 melhores scores e anexa o custo document_lengths[doc_id]
- **Refatorado:** nada da AV1 foi alterado; integração limpa

Configuração C3 → Candidatos

entrega Candidate(doc_id, score, tokens) para os 5 métodos

Linhas de base: topk_av1 e brute_force

top-5 sem limite (k = 5) · oráculo exato (2^n, n ≤ 20)

- **topk_av1:** pega os 5 primeiros do merge_sort ignorando W; em F4 (W = 4) gasta **11 tokens** (teste T06)
- **brute_force:** testa os 2^n subconjuntos (até 20 itens) para validar o DP; acerta F7, F9 e barra F8 (teste T05)
- **Três gulosos (Pacote 2):** completam as linhas de base respeitando W em Θ(n log n)

51/90 estouros · 30/30 exatos

topk_av1 estourou W em 51/90 casos; DP bateu 100% com a força bruta

Testes automatizados em **tests/test_p3_referencias.py**: **T05** (força bruta: F7, F8, F9) e **T06** (topk_av1: F4, F5)
Operações contabilizadas (**ops**): comparações do merge_sort no topk_av1 e 2^n na força bruta